Lesson End Project
Creating a React App with All Hooks

Project agenda: To develop a React application using Vite that demonstrates the use of all major React hooks, including state, effect, reducer, context, and custom hooks. The project will focus on modular architecture, global state management, asynchronous data flows, and real-time updates, ensuring reusable logic, maintainability, and an interactive user experience.

Description: You are tasked with building a React application that integrates all major React hooks to showcase their practical use in a modular and scalable project. This includes managing state with useState, side effects with useEffect, complex logic with useReducer, shared state using useContext, and reusable logic through custom hooks. The application will also feature real-time data updates, asynchronous data fetching, and a global task manager to demonstrate advanced state and data handling techniques.

Tools required: Visual Studio Code and Node.js

Prerequisites: None

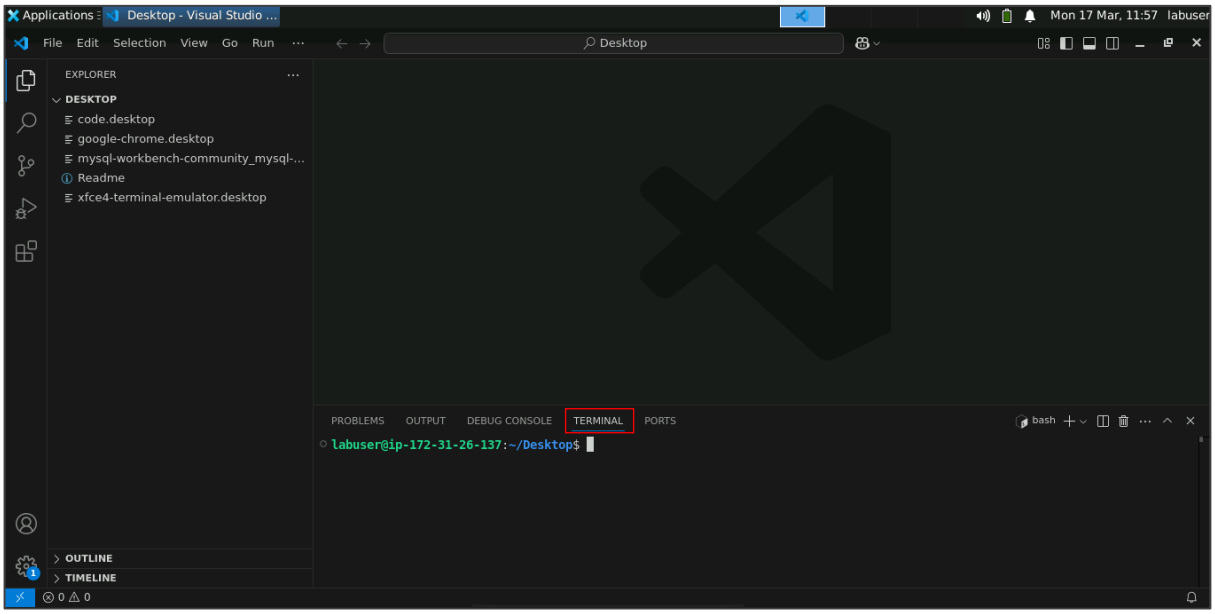
Expected deliverables: A fully functional React application built with Vite, demonstrating the use of all major React hooks. The application will include modular components, real-time number generation, a global task manager using context and reducer, a reusable counter hook, and asynchronous data fetching. It will showcase proper project structure, maintainable code, and interactive UI elements, serving as a comprehensive reference for applying React hooks in real-world scenarios.

Steps to be followed:

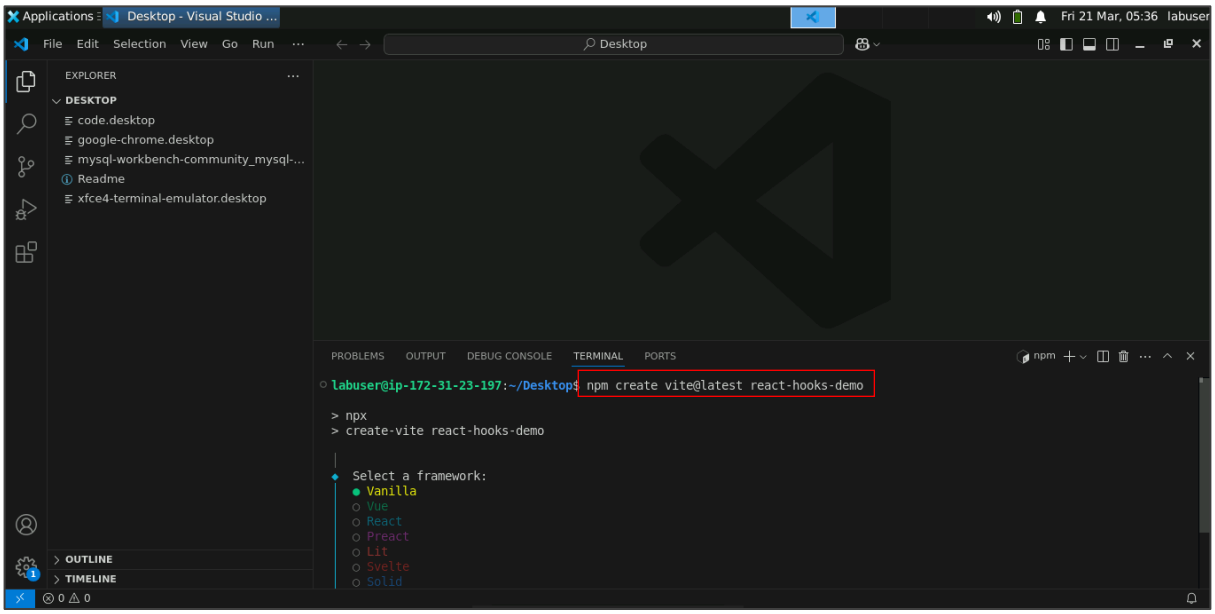
1. Set up a new React project using Vite
2. Structure the application
3. Implement core hooks and custom hooks
4. Build reusable UI components
5. Set up App component
6. Update main.jsx
7. Run and verify the application

Step 1: Set up a new React project using Vite

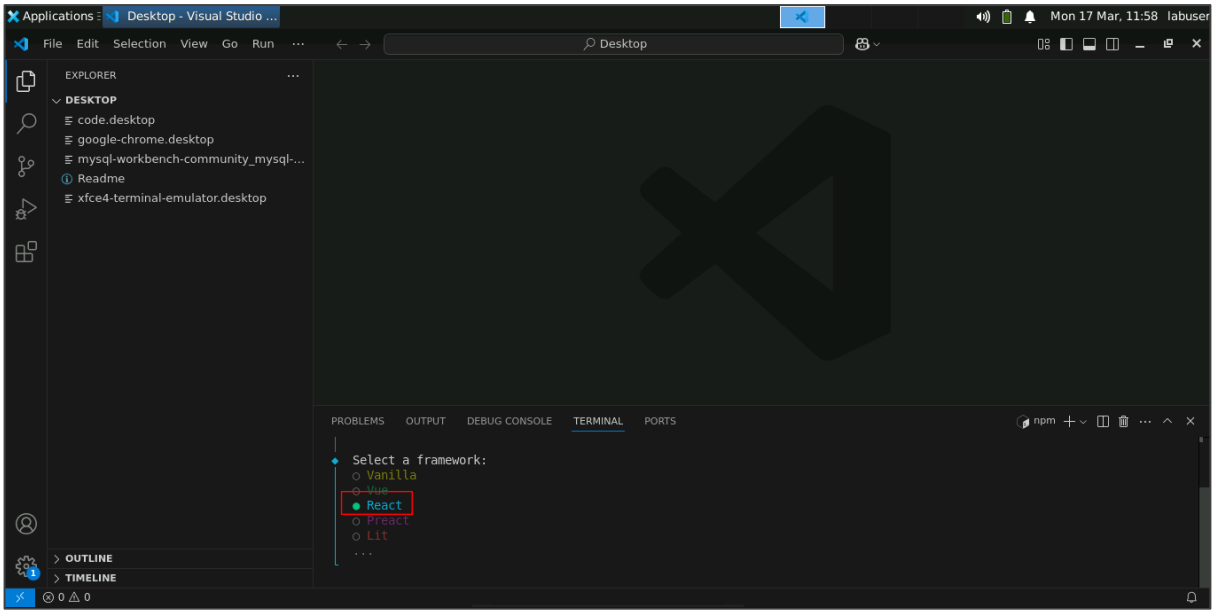
1.1 Navigate to VS Code and open the terminal



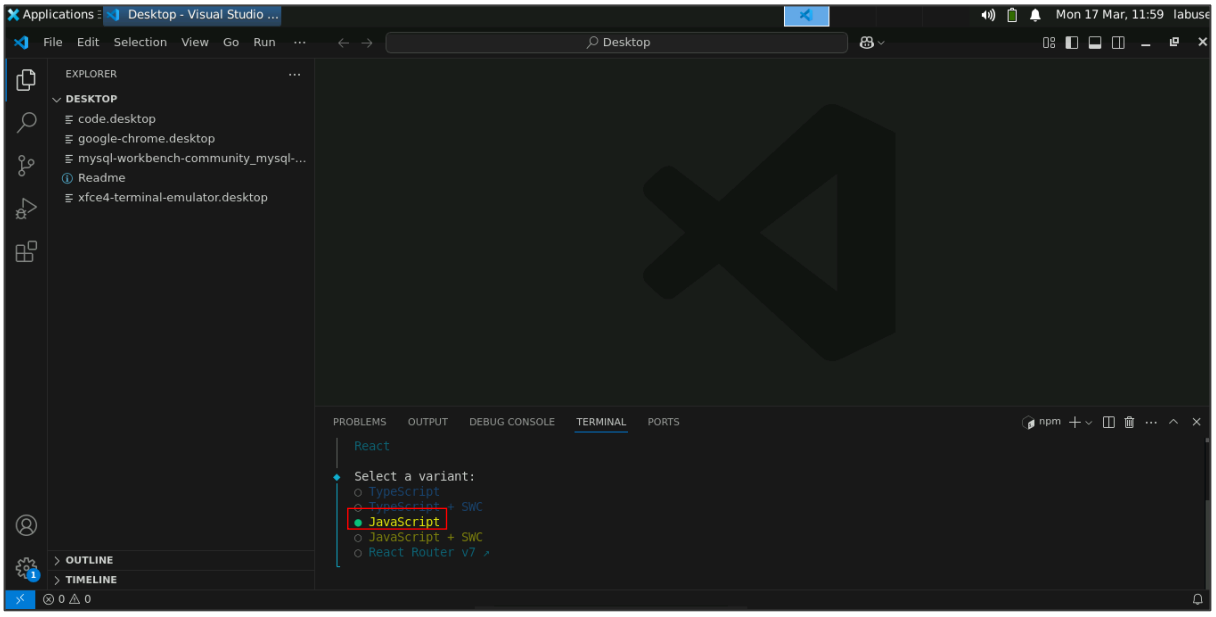
1.2 Run the following command to create a new React project using Vite:
npm create vite@latest react-hooks-demo



1.3 Select **React** and press enter

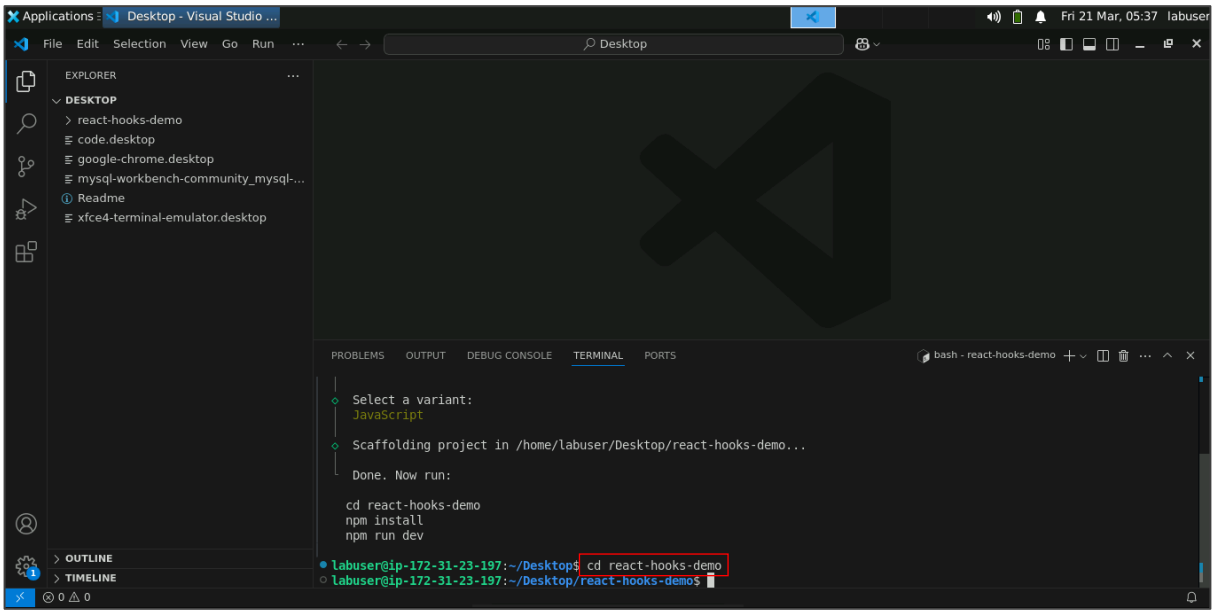


1.4 Select **JavaScript** and press enter



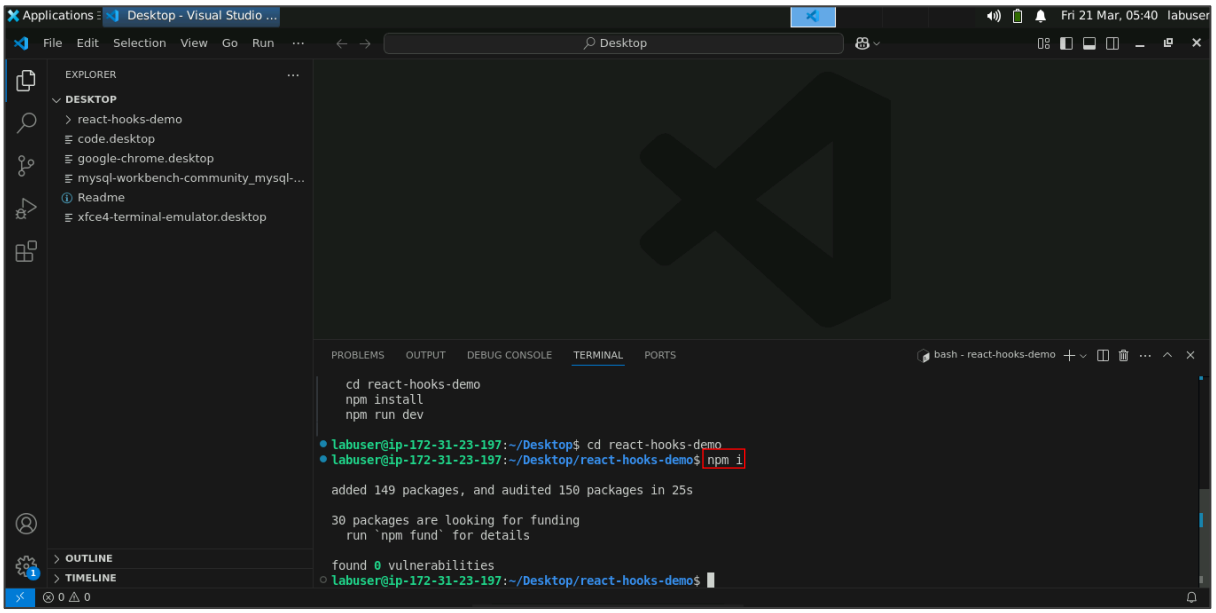
1.5 Run the following command to navigate to the project folder:

cd react-hooks-demo



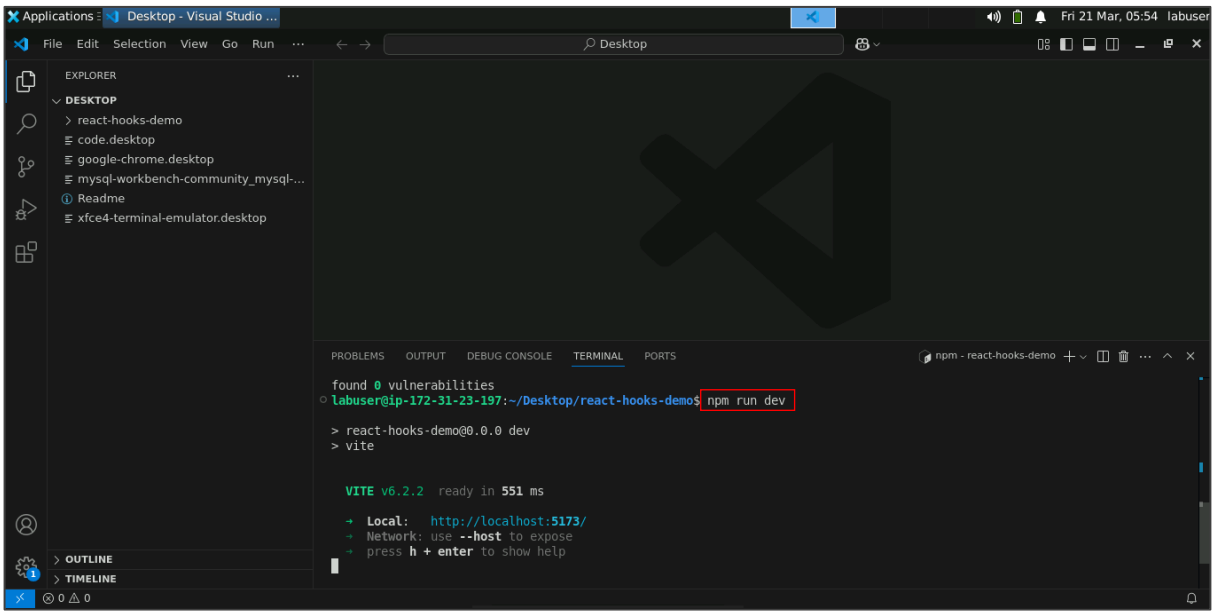
1.6 Run the following command to install dependencies:

npm i



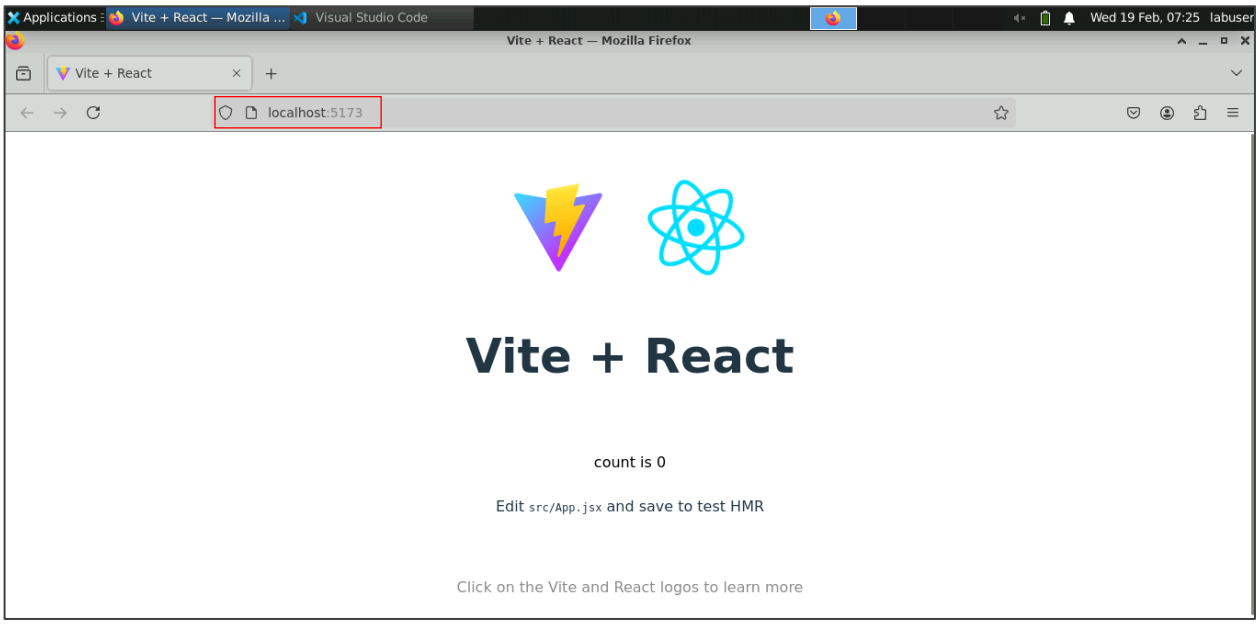
1.7 Run the following command to start the development server:

npm run dev



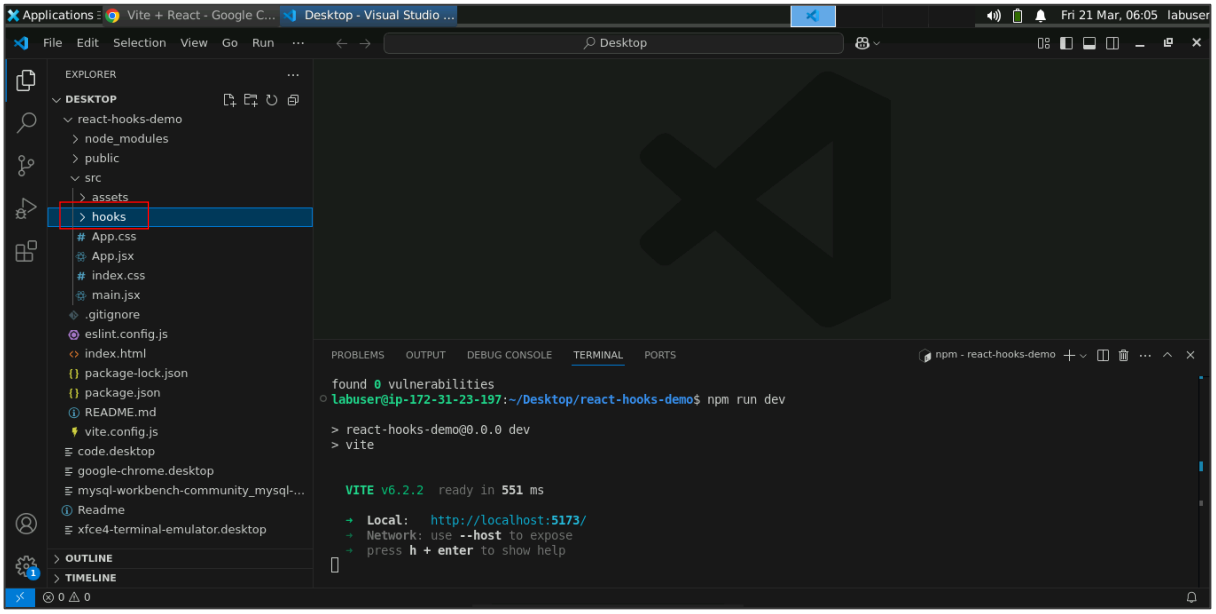
1.8 Navigate to the web browser and paste the link below:

http://localhost:5173

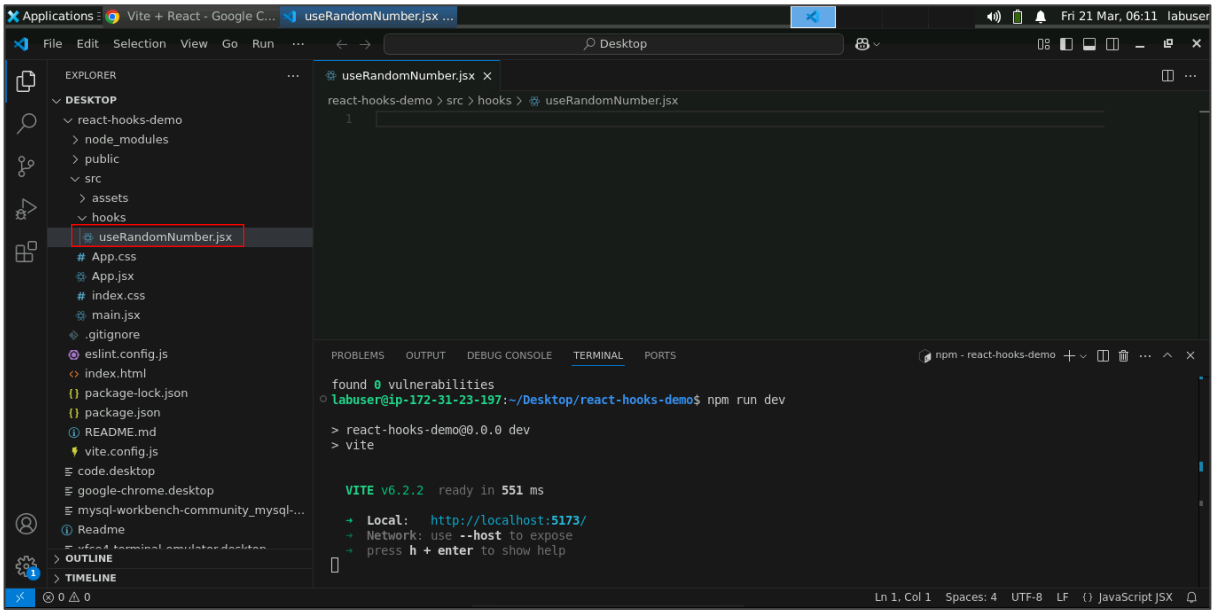


Step 2: Structure the application

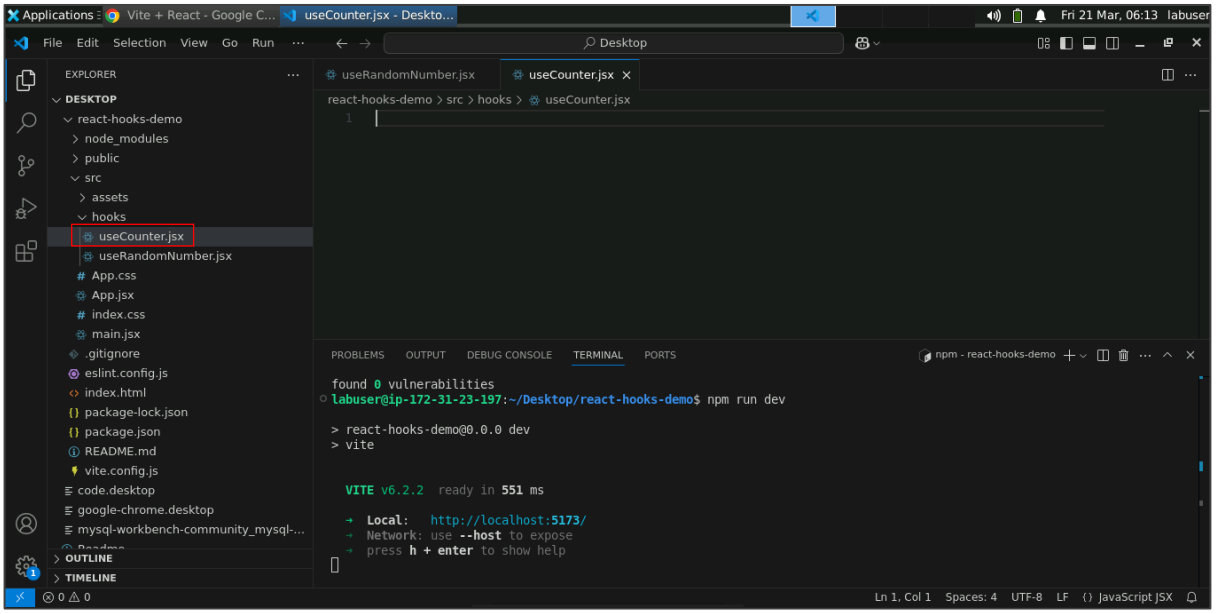
2.1 Inside the **src** folder, create a new folder named **hooks**



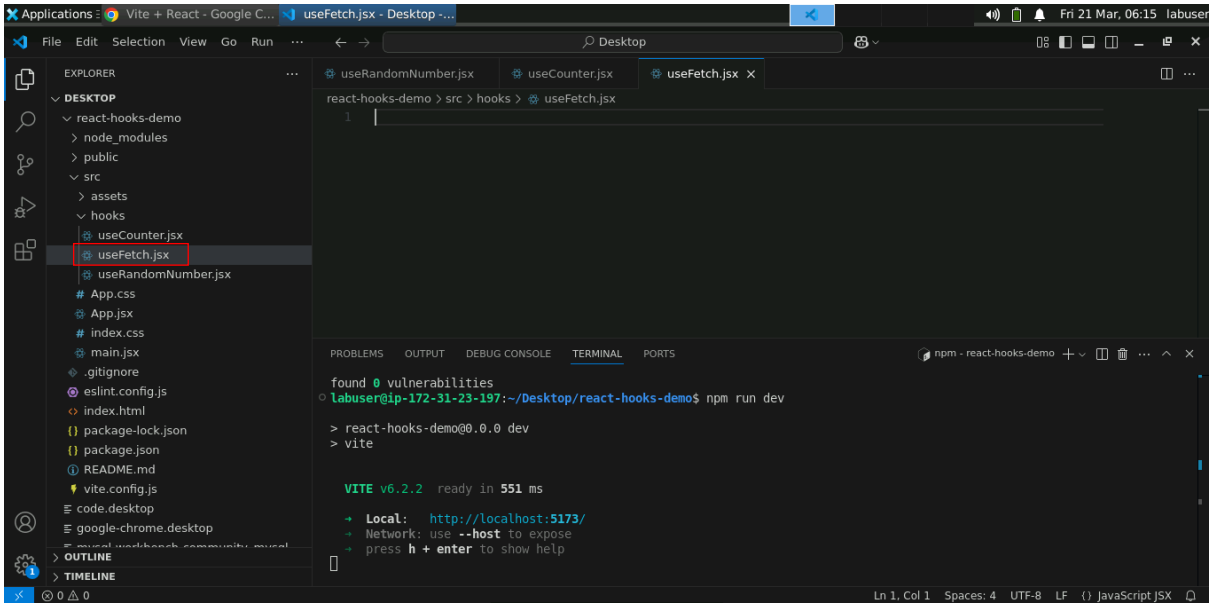
2.2 Inside the **hooks** folder, create a new file named **useRandomNumber.jsx**



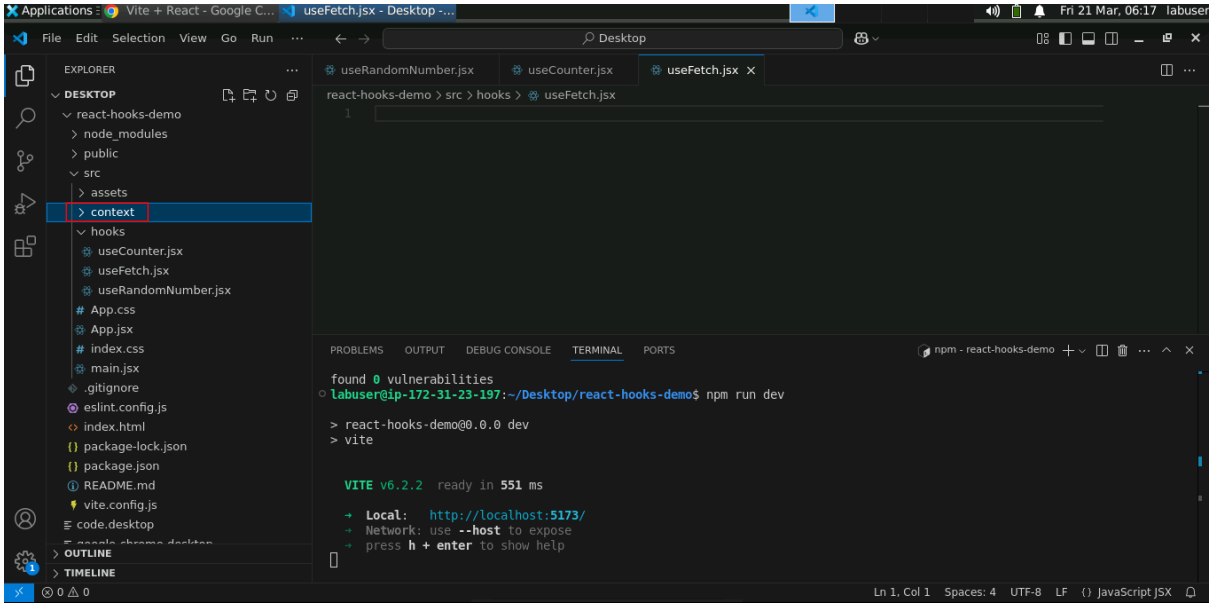
2.3 Inside the **hooks** folder, create a new file named **useCounter.jsx**



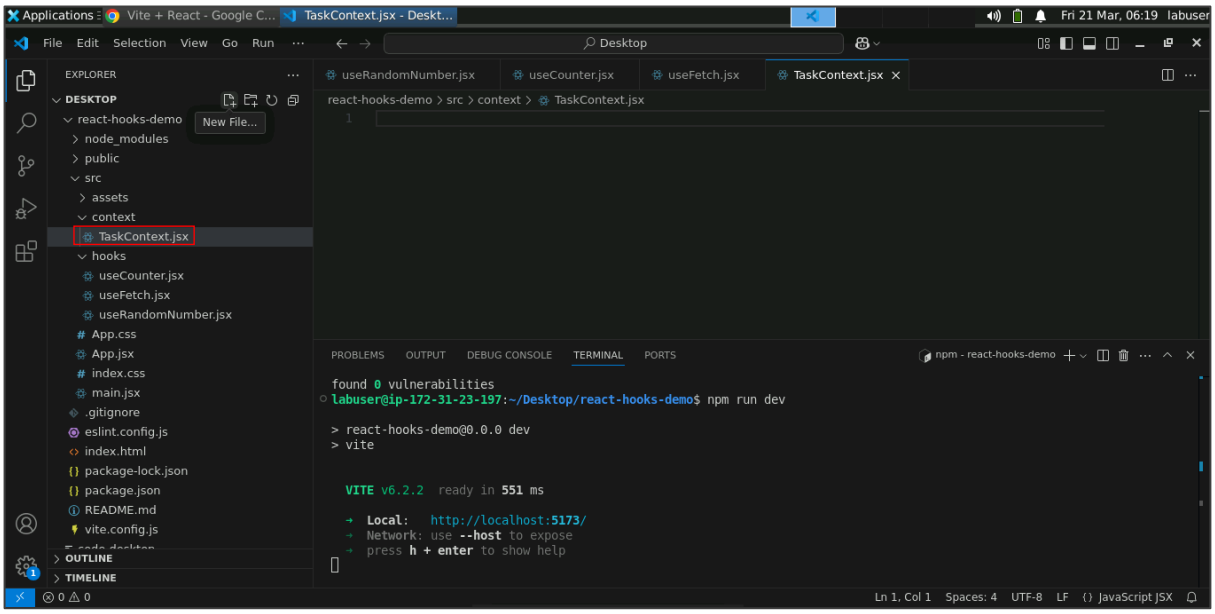
2.4 Inside the **hooks** folder, create a new file named **useFetch.jsx**



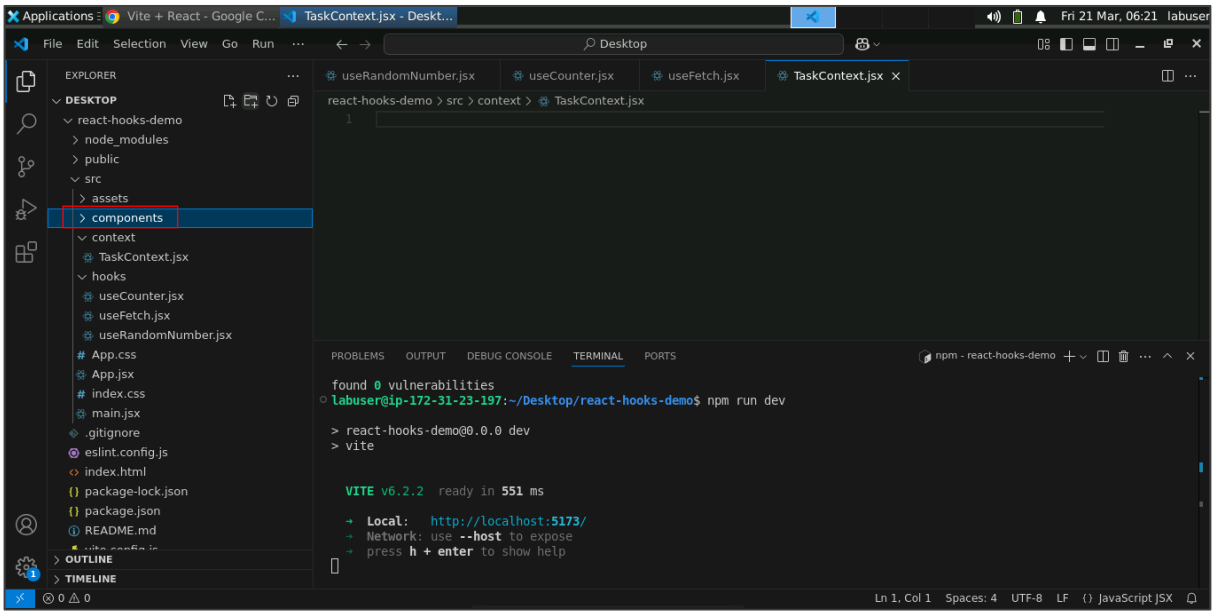
2.5 Inside the **src** folder, create a new folder named **context**



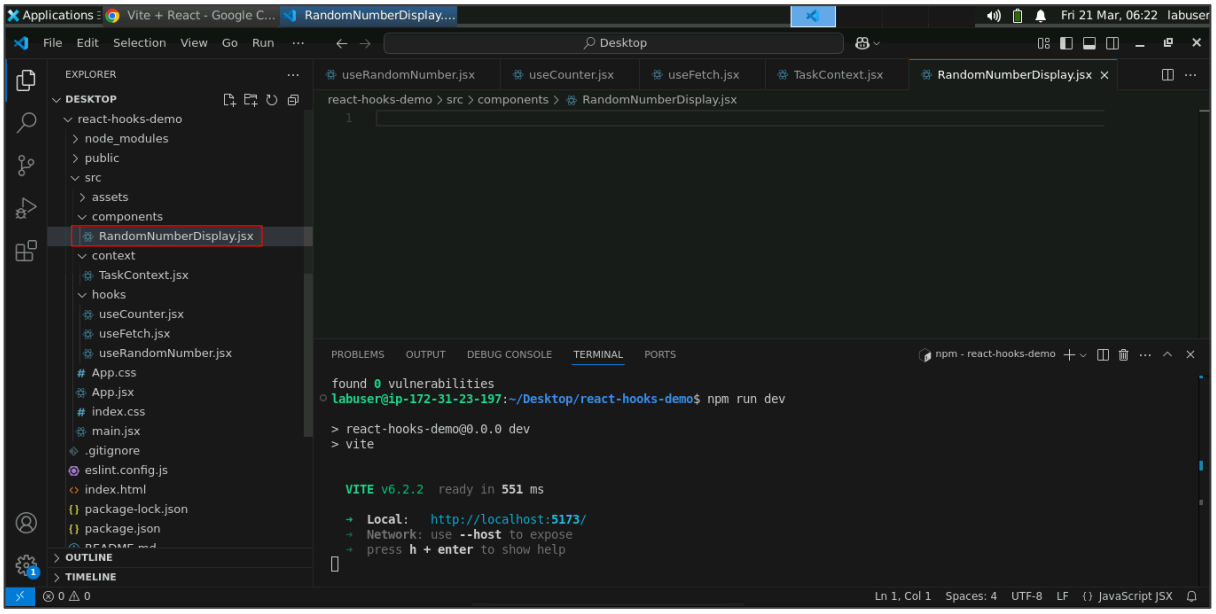
2.6 Inside the **context** folder, create a new file named **TaskContext.jsx**



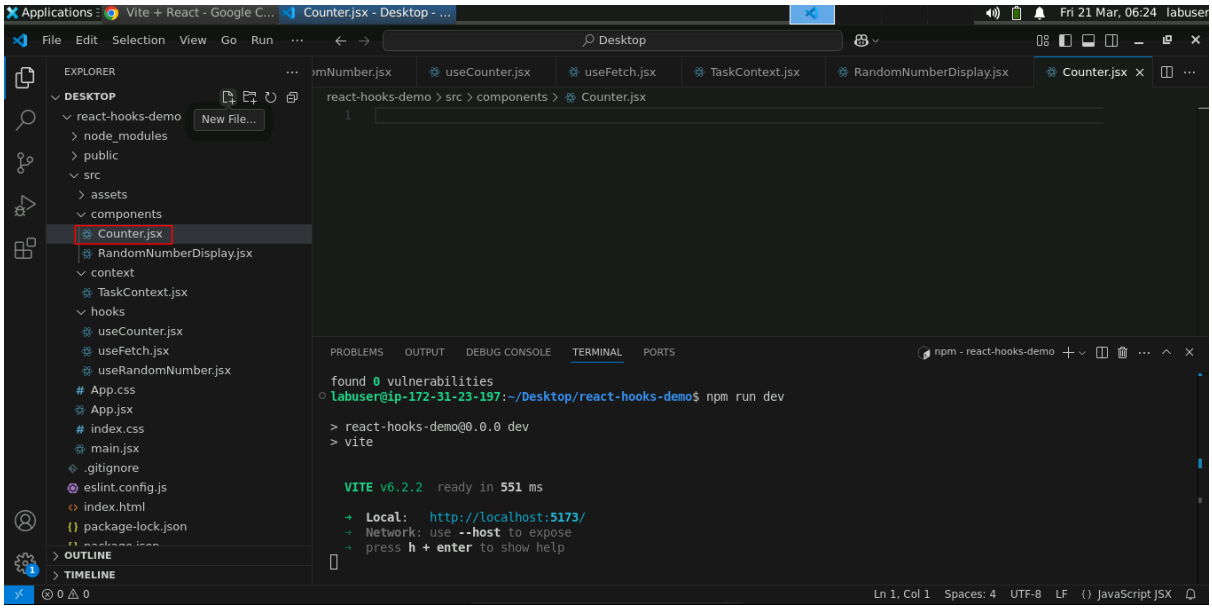
2.7 Inside the **src** folder, create a new folder named **components**



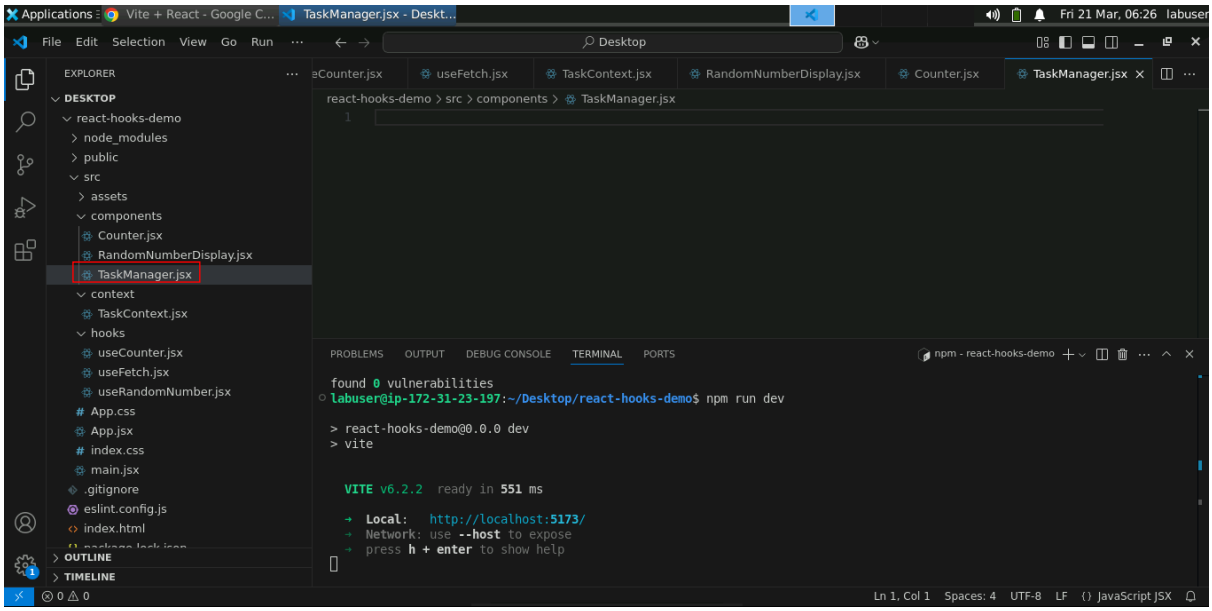
2.8 Inside the **components** folder, create a new file named **RandomNumberDisplay.jsx**



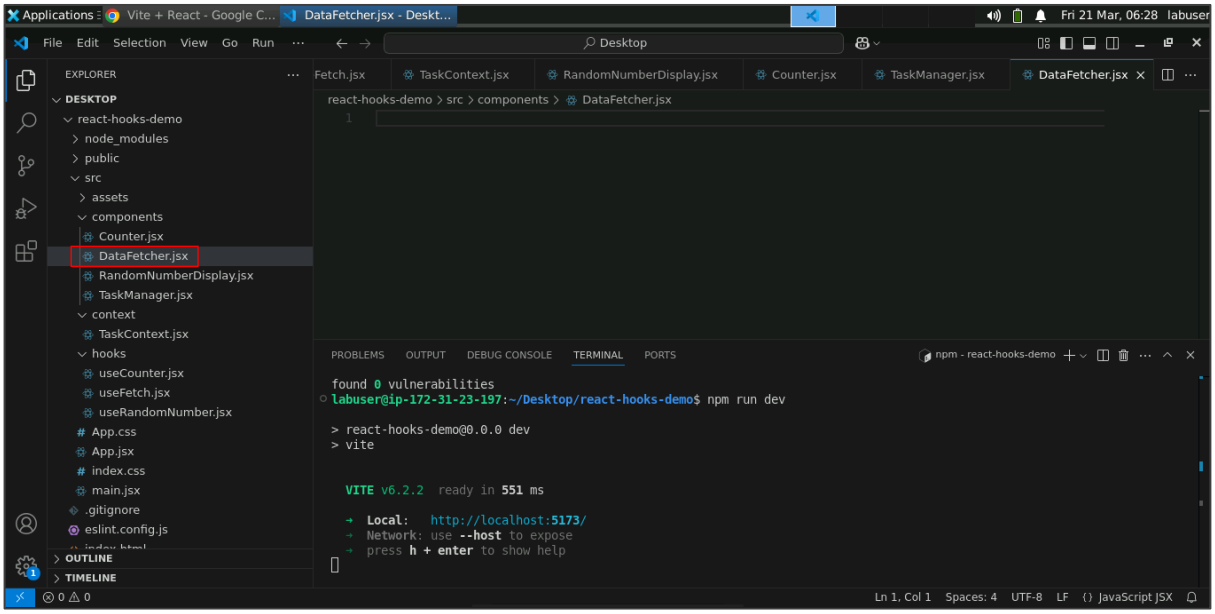
2.9 Inside the **components** folder, create a new file named **Counter.jsx**



2.10 Inside the **components** folder, create a new file named **TaskManager.jsx**



2.11 Inside the **components** folder, create a new file named **DataFetcher.jsx**

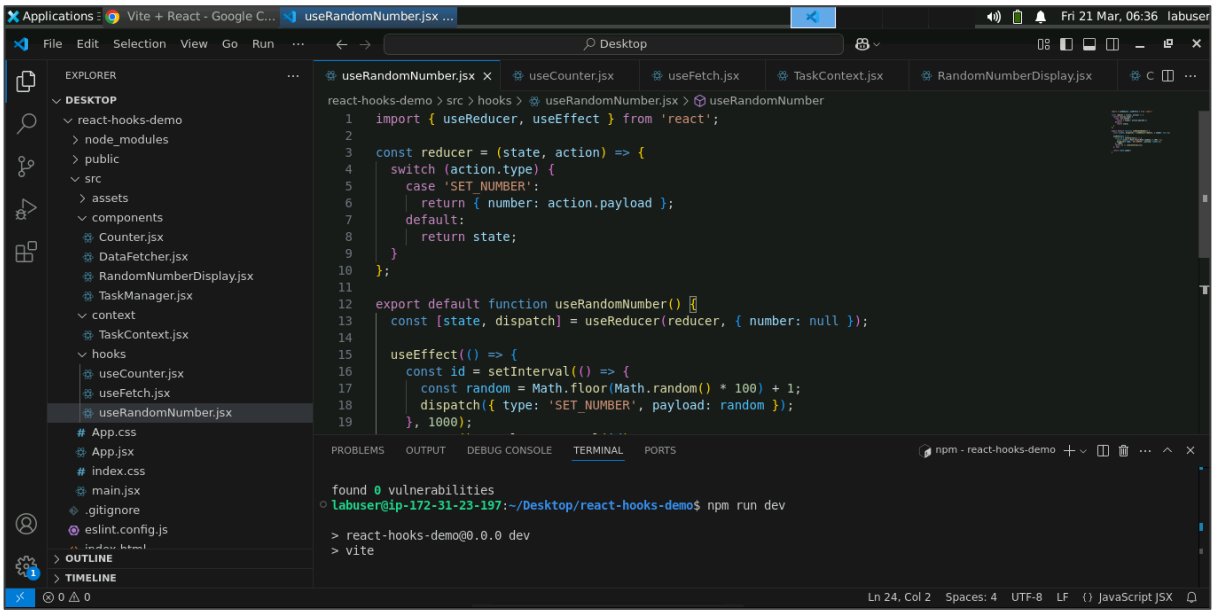


Step 3: Implement Custom Hooks and Context

3.1 Paste the code below in **useRandomNumber.jsx**:

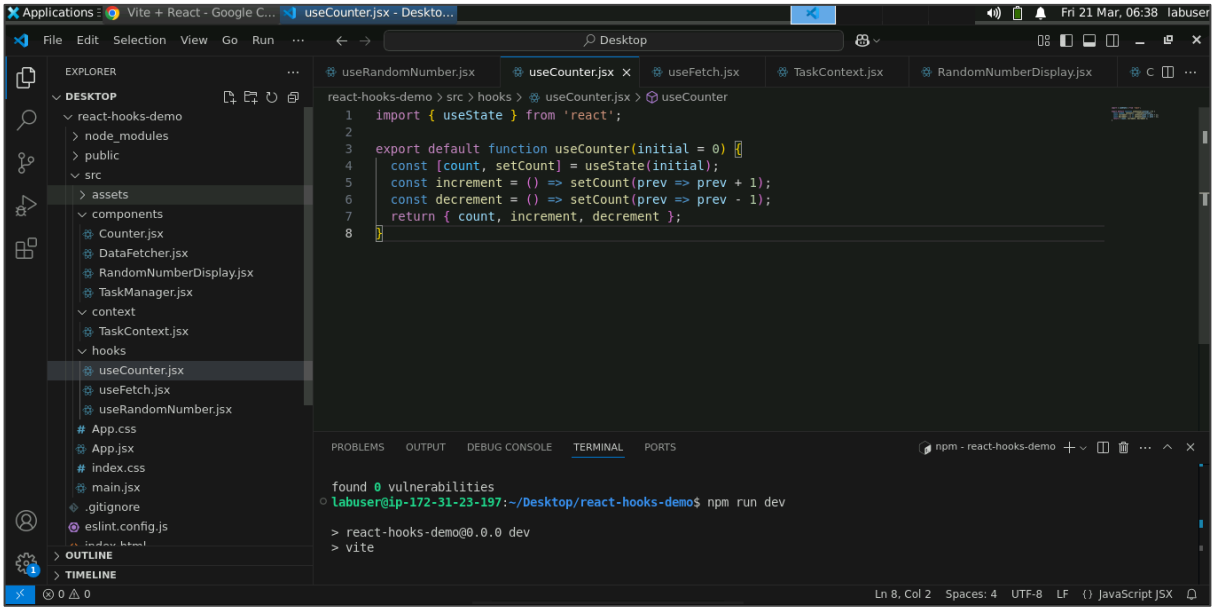
```
import { useReducer, useEffect } from 'react';

const reducer = (state, action) => {
  switch (action.type) {
    case 'SET_NUMBER':
      return { number: action.payload };
    default:
      return state;
  }
};

export default function useRandomNumber() {
  const [state, dispatch] = useReducer(reducer, { number: null });

  useEffect(() => {
    const id = setInterval(() => {
      const random = Math.floor(Math.random() * 100) + 1;
      dispatch({ type: 'SET_NUMBER', payload: random });
    }, 1000);
    return () => clearInterval(id);
  }, []);

  return state.number;
```

```
}
```



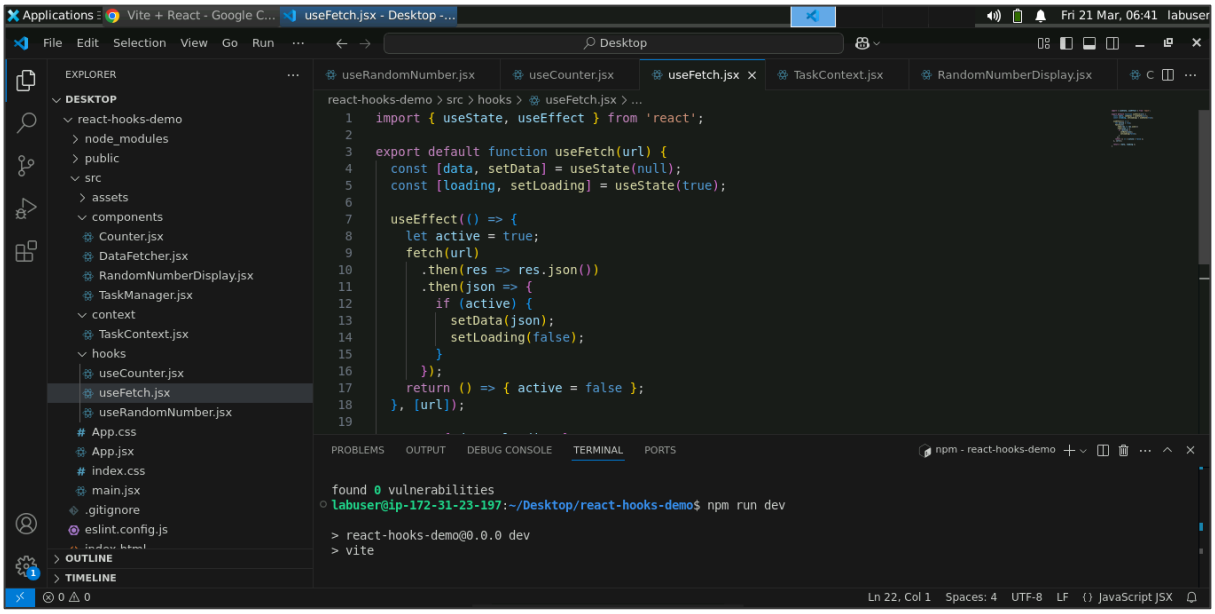
3.2 Paste the code below in **useCounter.jsx:**

```
import { useState } from 'react';

export default function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);
  const increment = () => setCount(prev => prev + 1);
  const decrement = () => setCount(prev => prev - 1);
  return { count, increment, decrement };
}
```



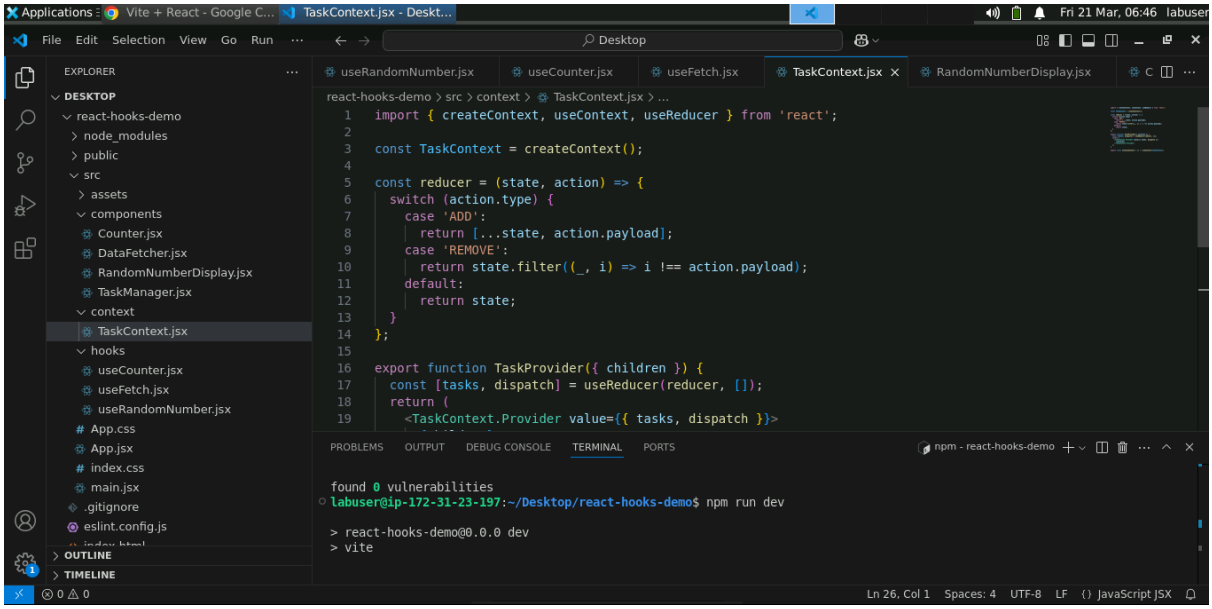
3.3 Paste the code below in **useFetch.jsx**:

```
import { useState, useEffect } from 'react';

export default function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let active = true;
    fetch(url)
      .then(res => res.json())
      .then(json => {
        if (active) {
          setData(json);
          setLoading(false);
        }
      });
    return () => { active = false };
  }, [url]);

  return { data, loading };
}
```



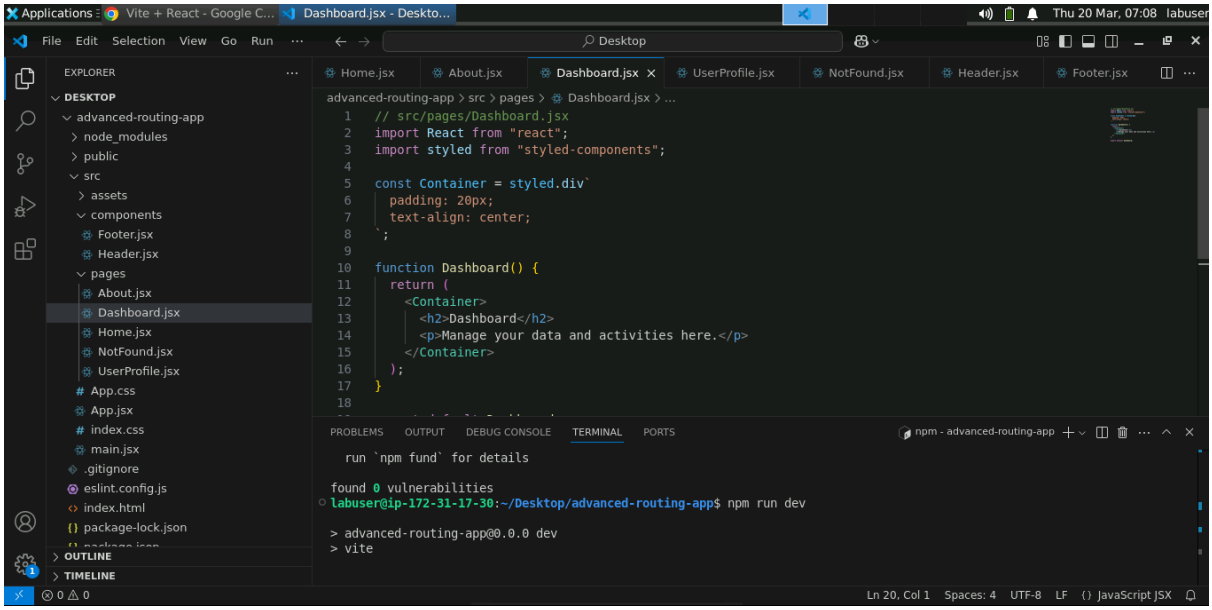
3.4 Paste the code below in **TaskContext.jsx**:

```
import { createContext, useContext, useReducer } from 'react';

const TaskContext = createContext();

const reducer = (state, action) => {
  switch (action.type) {
    case 'ADD':
      return [...state, action.payload];
    case 'REMOVE':
      return state.filter((_, i) => i !== action.payload);
    default:
      return state;
  }
};

export function TaskProvider({ children }) {
  const [tasks, dispatch] = useReducer(reducer, []);
  return (
    <TaskContext.Provider value={{ tasks, dispatch }}>
      {children}
    </TaskContext.Provider>
  );
}

export const useTaskContext = () => useContext(TaskContext);
```



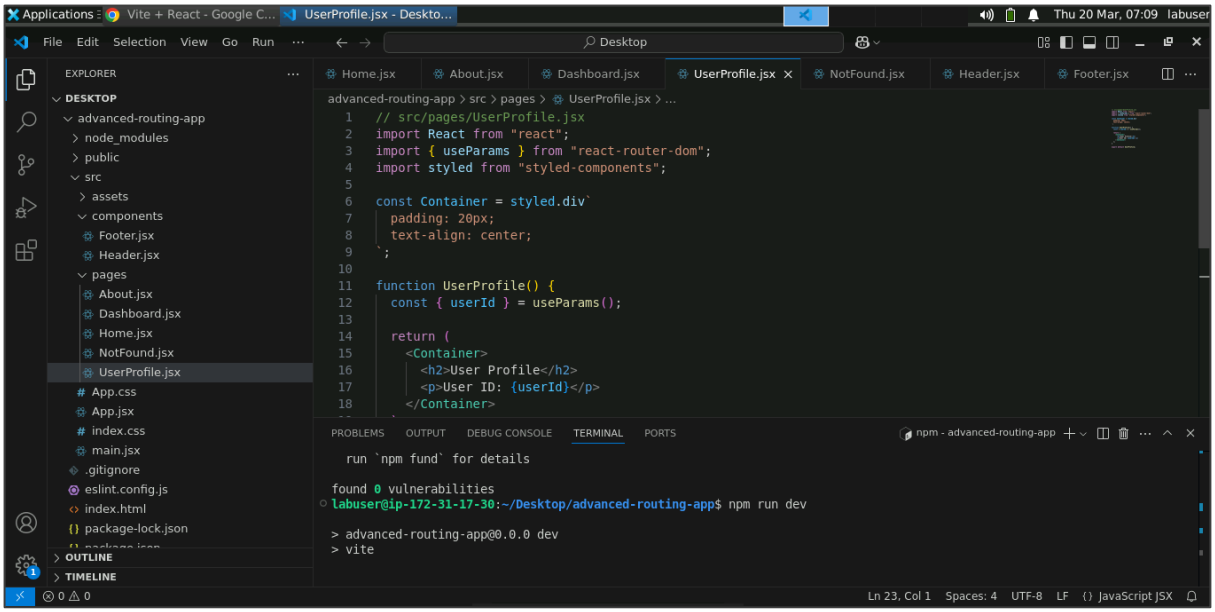
3.5 Paste the code below in **Dashboard.jsx**:

```
// src/pages/Dashboard.jsx
import React from "react";
import styled from "styled-components";

const Container = styled.div`
  padding: 20px;
  text-align: center;
`;

function Dashboard() {
  return (
    <Container>
      <h2>Dashboard</h2>
      <p>Manage your data and activities here.</p>
    </Container>
  );
}

export default Dashboard;
```



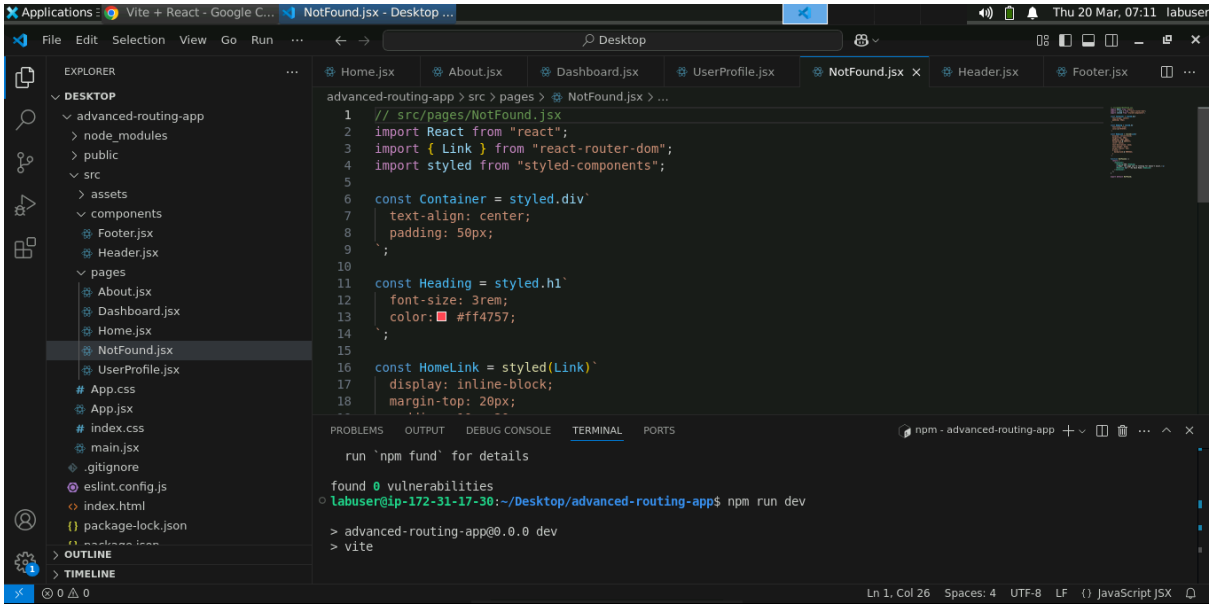
3.6 Paste the code below in **UserProfile.jsx**:

```
// src/pages/UserProfile.jsx
import React from "react";
import { useParams } from "react-router-dom";
import styled from "styled-components";

const Container = styled.div`
  padding: 20px;
  text-align: center;
`;

function UserProfile() {
  const { userId } = useParams();

  return (
    <Container>
      <h2>User Profile</h2>
      <p>User ID: {userId}</p>
    </Container>
  );
}

export default UserProfile;
```



3.7 Paste the code below in **NotFound.jsx**:

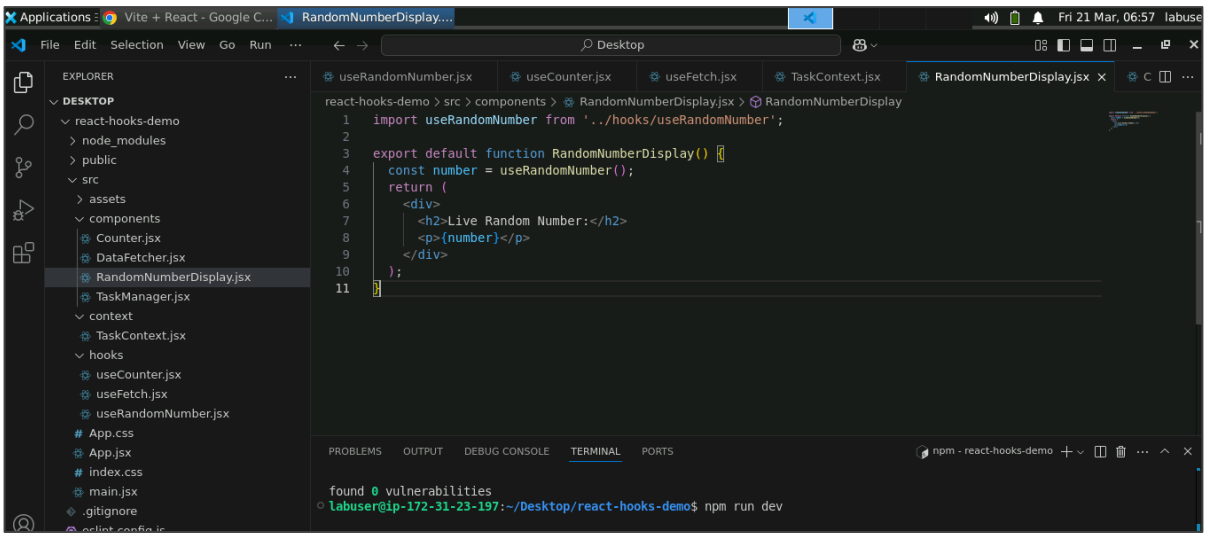
```
// src/pages/NotFound.jsx
import React from "react";
import { Link } from "react-router-dom";
import styled from "styled-components";

const Container = styled.div`
  text-align: center;
  padding: 50px;
`;

const Heading = styled.h1`
  font-size: 3rem;
  color: #ff4757;
`;

const HomeLink = styled(Link)`
  display: inline-block;
  margin-top: 20px;
  padding: 10px 20px;
  background: #007bff;
  color: white;
  text-decoration: none;
  font-weight: bold;
  border-radius: 5px;
  &:hover {
    background: #0056b3;
  }
`;

function NotFound() {
  return (
    <Container>
      <Heading>404</Heading>
      <p>Oops! The page you're looking for doesn't exist.</p>
      <HomeLink to="/">Go Back Home</HomeLink>
    </Container>
  );
}

export default NotFound;
```



Step 4: Build reusable UI components

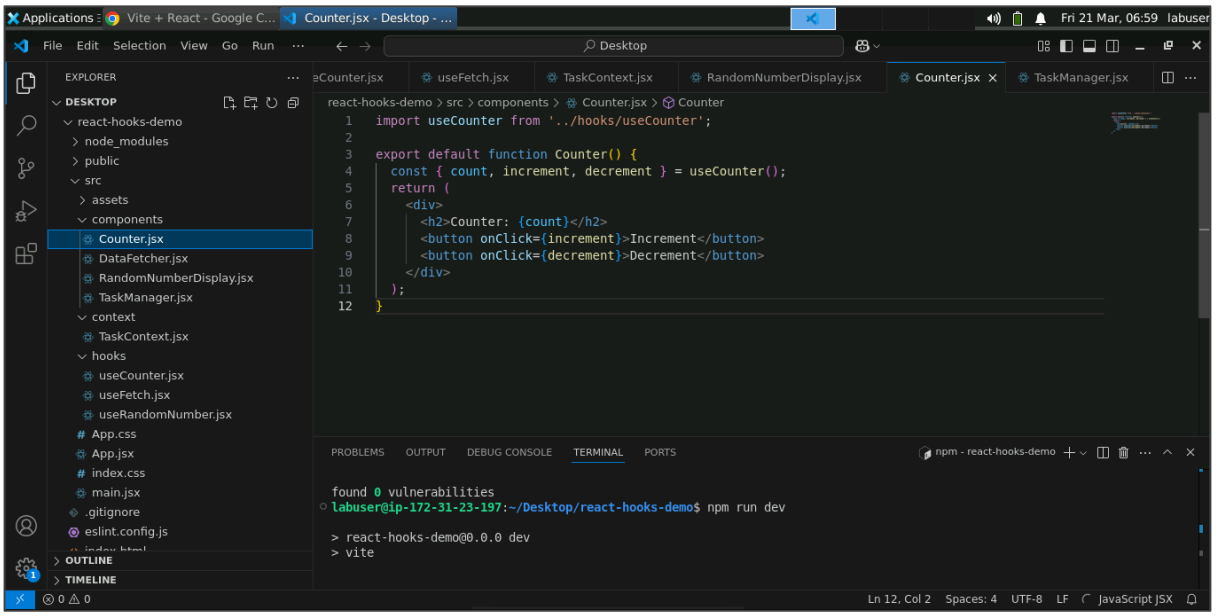
4.1 Paste the code below in **RandomNumberDisplay.jsx**:

```
import useRandomNumber from '../hooks/useRandomNumber';

export default function RandomNumberDisplay() {
  const number = useRandomNumber();
  return (
    <div>
      <h2>Live Random Number:</h2>
      <p>{number}</p>
    </div>
  );
}
```



4.2 Paste the code below in **Counter.jsx**:

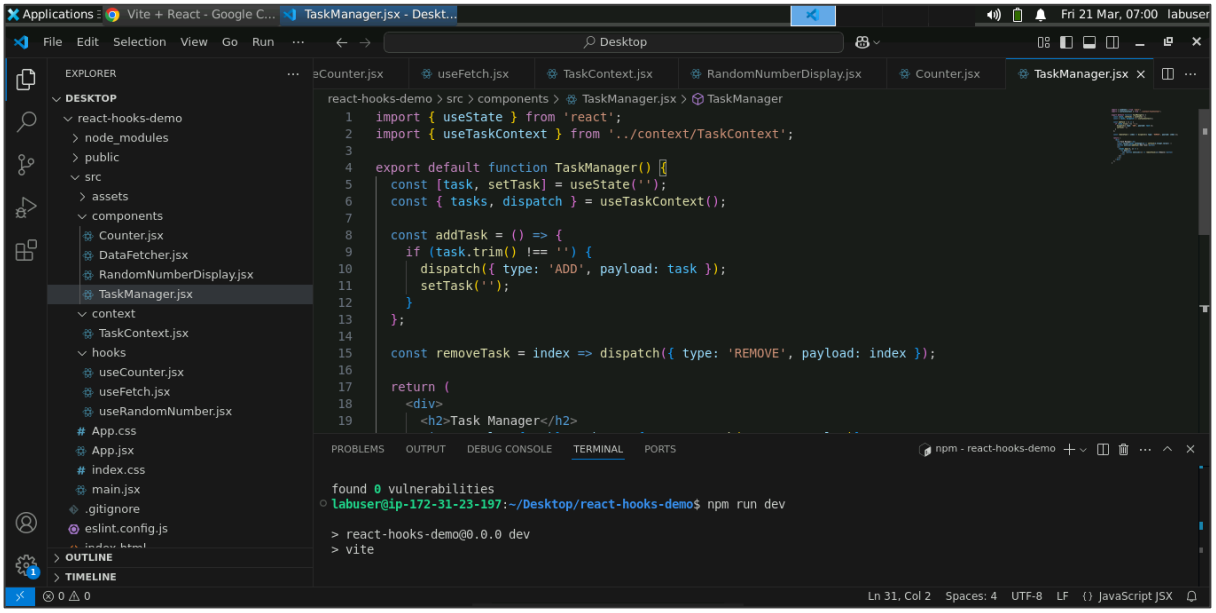
```
import useCounter from '../hooks/useCounter';

export default function Counter() {
  const { count, increment, decrement } = useCounter();
  return (
    <div>
      <h2>Counter: {count}</h2>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
}
```



4.3 Paste the code below in **TaskManager.jsx**:

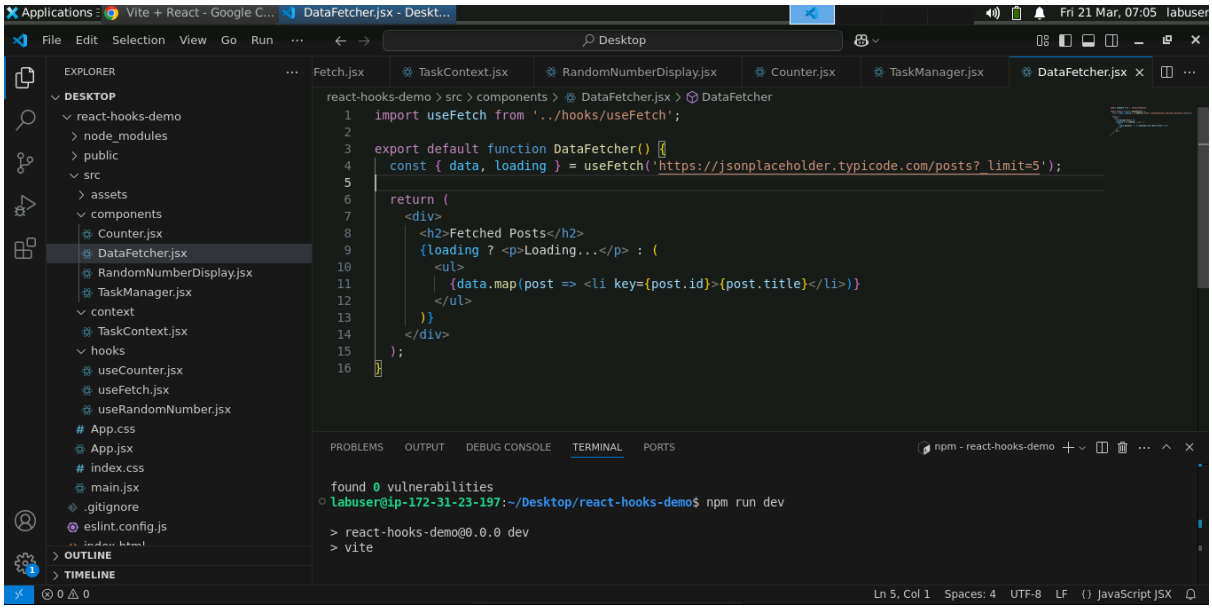
```
import { useState } from 'react';
import { useTaskContext } from '../context/TaskContext';

export default function TaskManager() {
  const [task, setTask] = useState('');
  const { tasks, dispatch } = useTaskContext();

  const addTask = () => {
    if (task.trim() !== '') {
      dispatch({ type: 'ADD', payload: task });
```

```
    setTask('');
  }
};

const removeTask = index => dispatch({ type: 'REMOVE', payload: index });

return (
  <div>
    <h2>Task Manager</h2>
    <input value={task} onChange={e => setTask(e.target.value)} />
    <button onClick={addTask}>Add Task</button>
    <ul>
      {tasks.map((t, i) => (
        <li key={i}>
          {t} <button onClick={() => removeTask(i)}>Remove</button>
        </li>
      ))}
    </ul>
  </div>
);
}
```



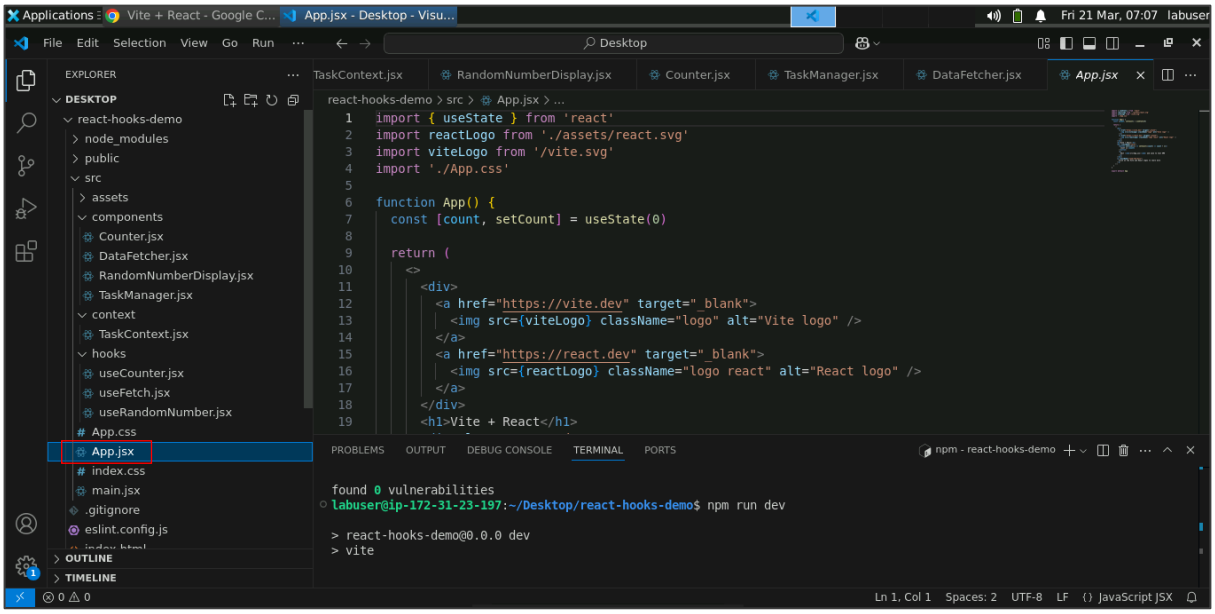
4.4 Paste the code below in **DataFetcher.jsx**:

```
import useFetch from '../hooks/useFetch';

export default function DataFetcher() {
  const { data, loading } =
useFetch('https://jsonplaceholder.typicode.com/posts?_limit=5');

  return (
    <div>
      <h2>Fetched Posts</h2>
      {loading ? <p>Loading...</p> : (
        <ul>
          {data.map(post => <li key={post.id}>{post.title}</li>)}
        </ul>
      )}
    </div>
  );
}
```



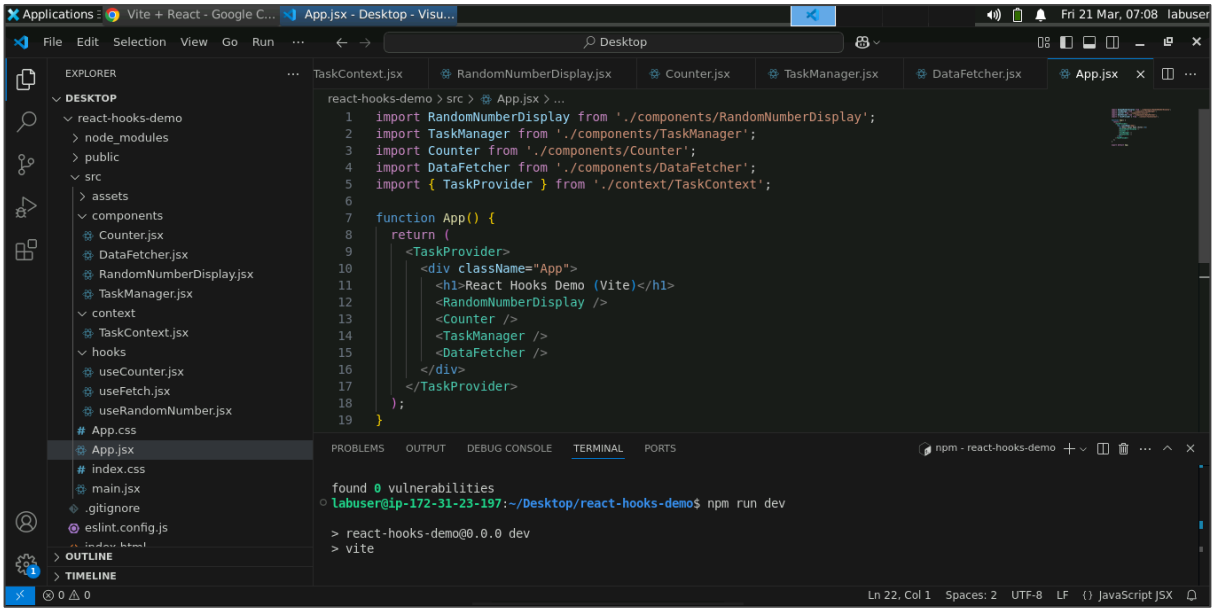
Step 5: Set up the App component

5.1 Open the **App.jsx** file inside the **src** folder



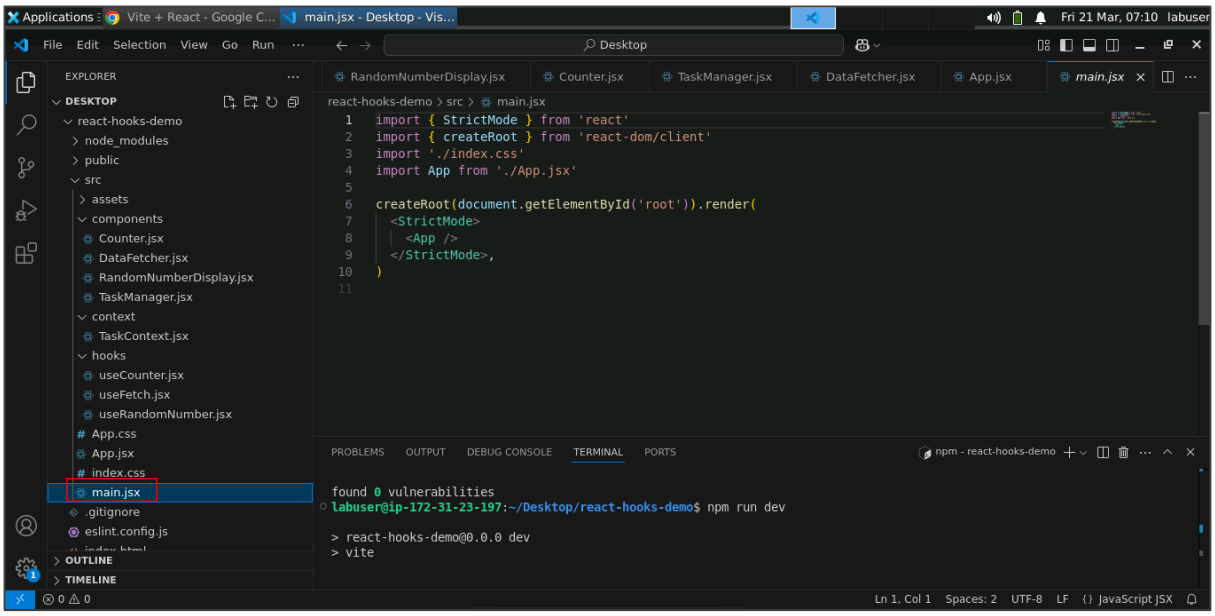
5.2 Paste the code below to include the new components in **App.jsx**:

```
import RandomNumberDisplay from './components/RandomNumberDisplay';
import TaskManager from './components/TaskManager';
import Counter from './components/Counter';
import DataFetcher from './components/DataFetcher';
import { TaskProvider } from './context/TaskContext';

function App() {
  return (
    <TaskProvider>
      <div className="App">
        <h1>React Hooks Demo (Vite)</h1>
        <RandomNumberDisplay />
        <Counter />
        <TaskManager />
        <DataFetcher />
      </div>
    </TaskProvider>
  );
}

export default App;
```



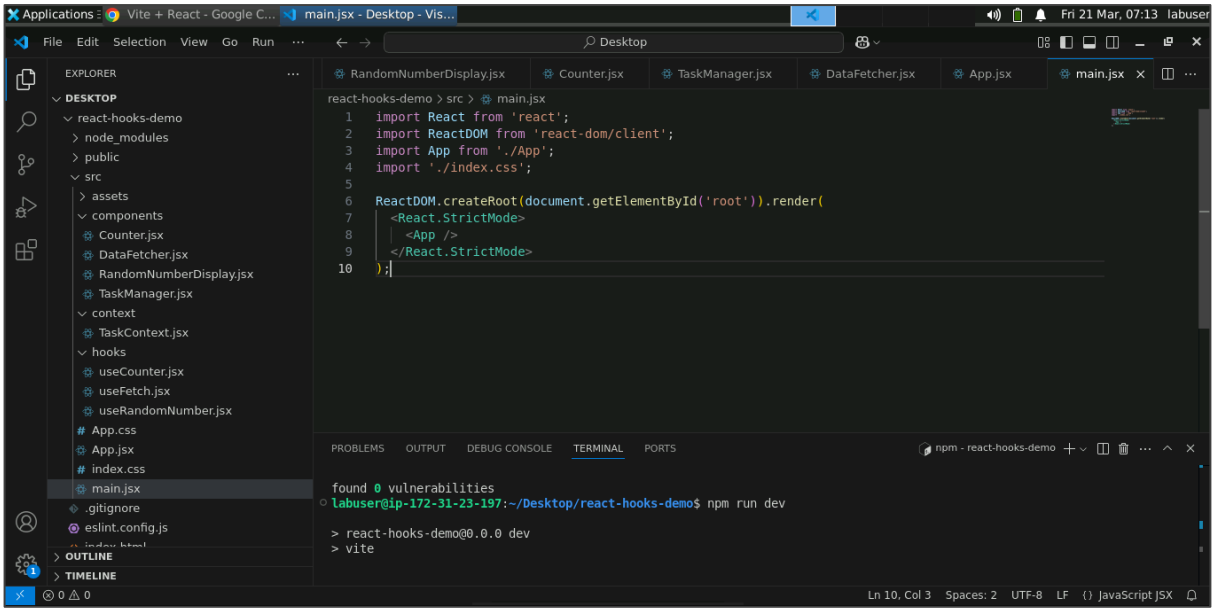
Step 6: Update main.jsx

6.1 Open the **main.jsx** file inside the **src** folder



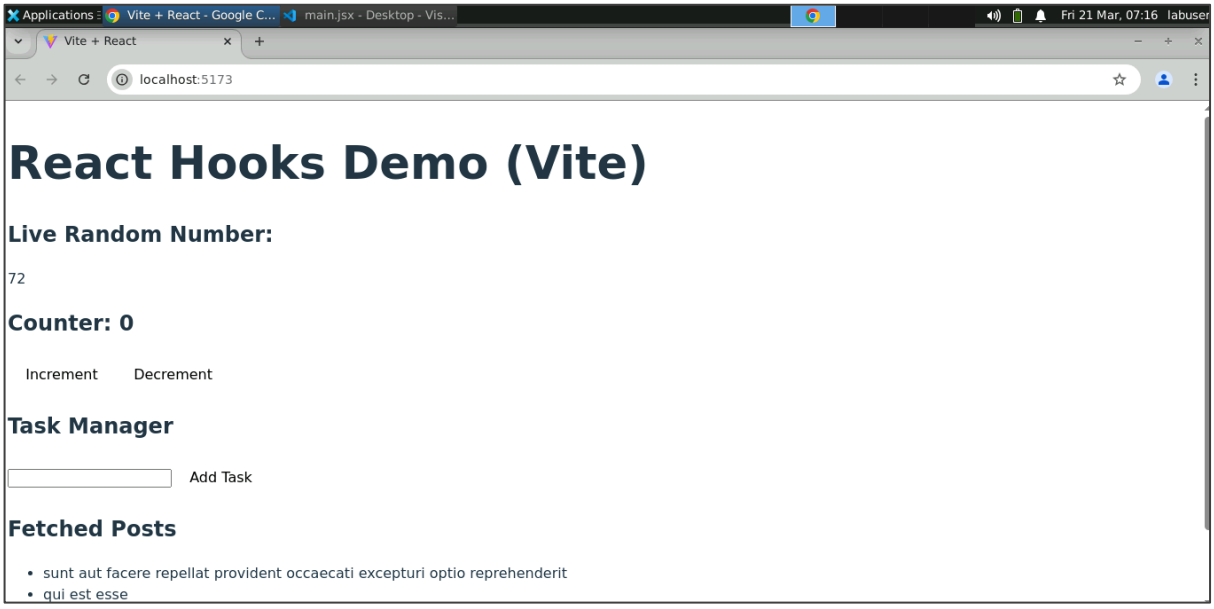
6.2 Paste the code below to include the new components in **main.jsx**:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
 <React.StrictMode>
  <App />
 </React.StrictMode>
);
```

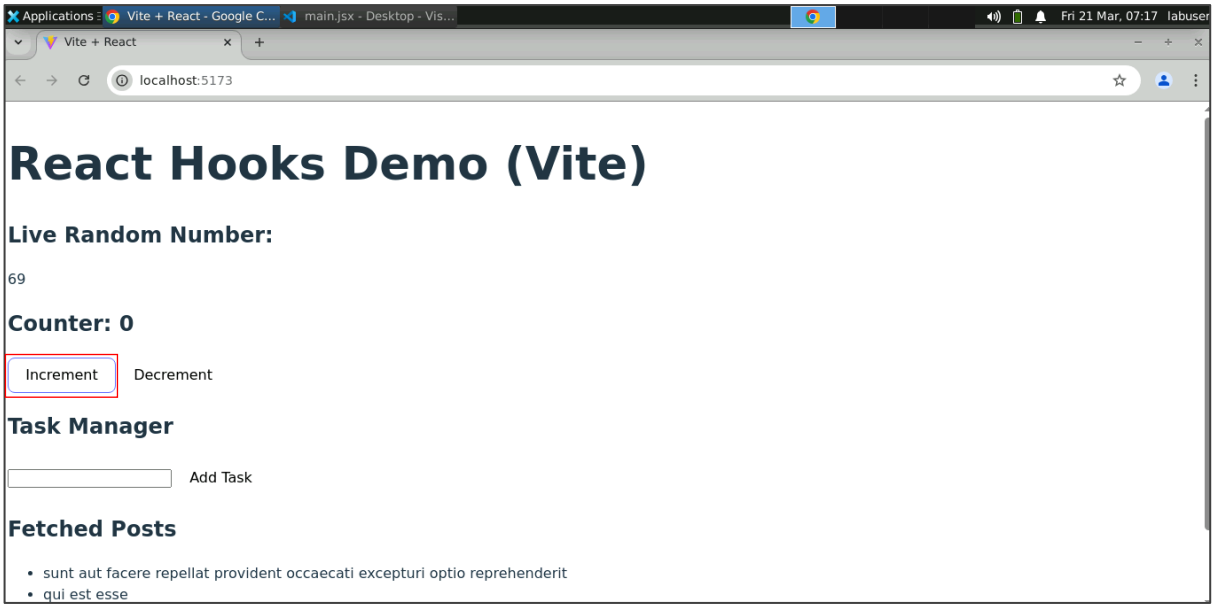


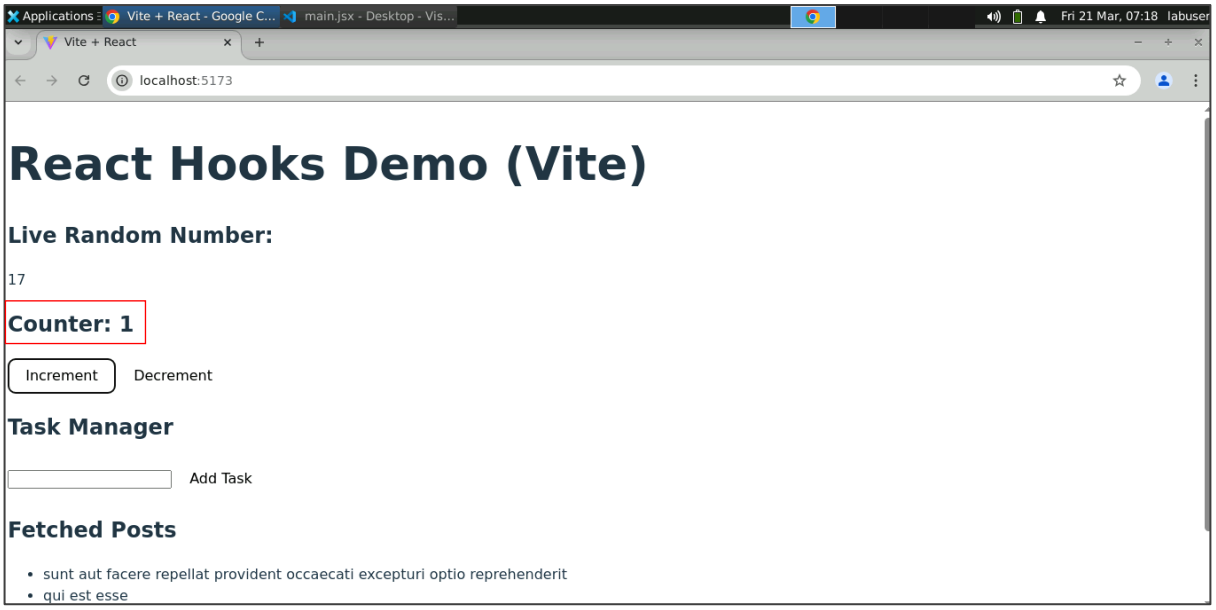
Step 7: Run and verify the application

7.1 Navigate to the browser and refresh the webpage

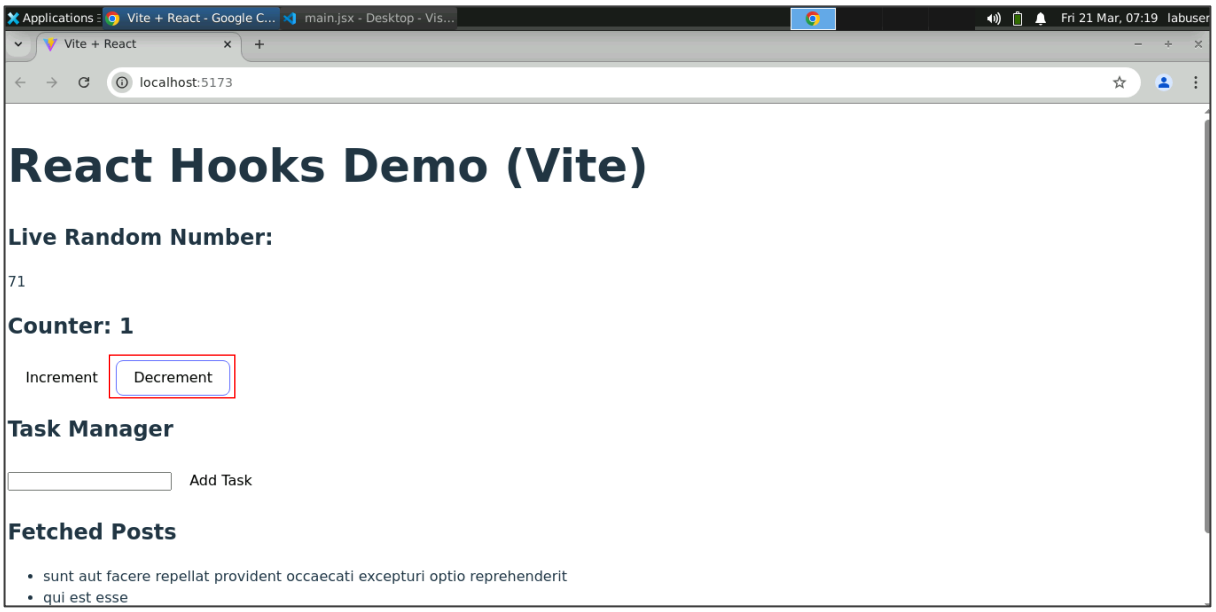


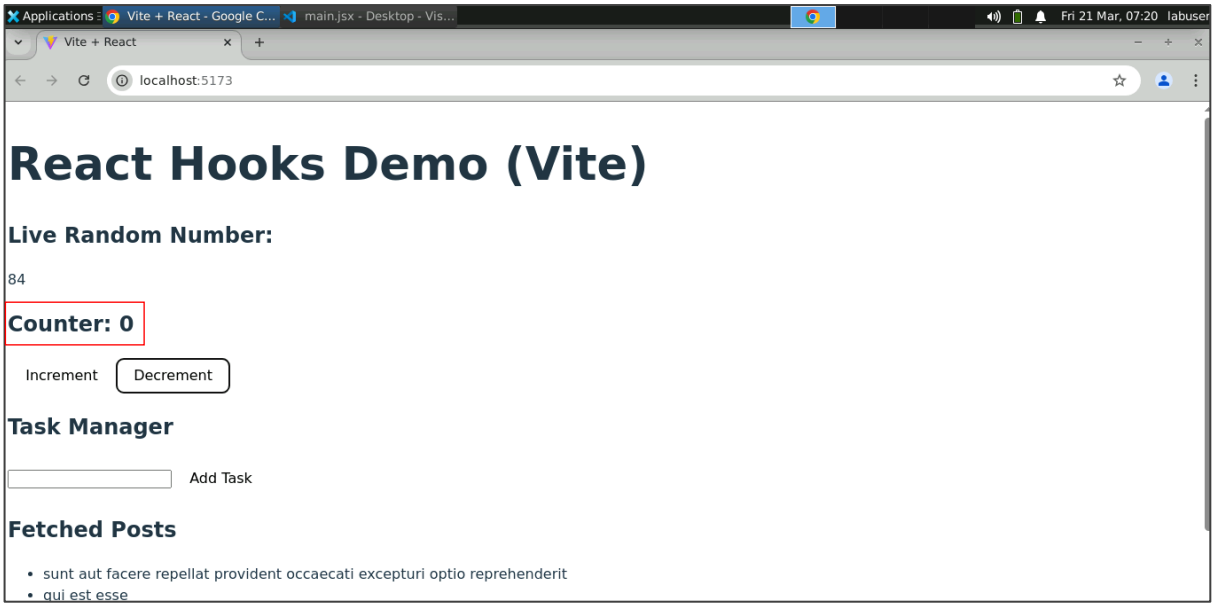
7.2 Click **Increment** to increase the count by 1



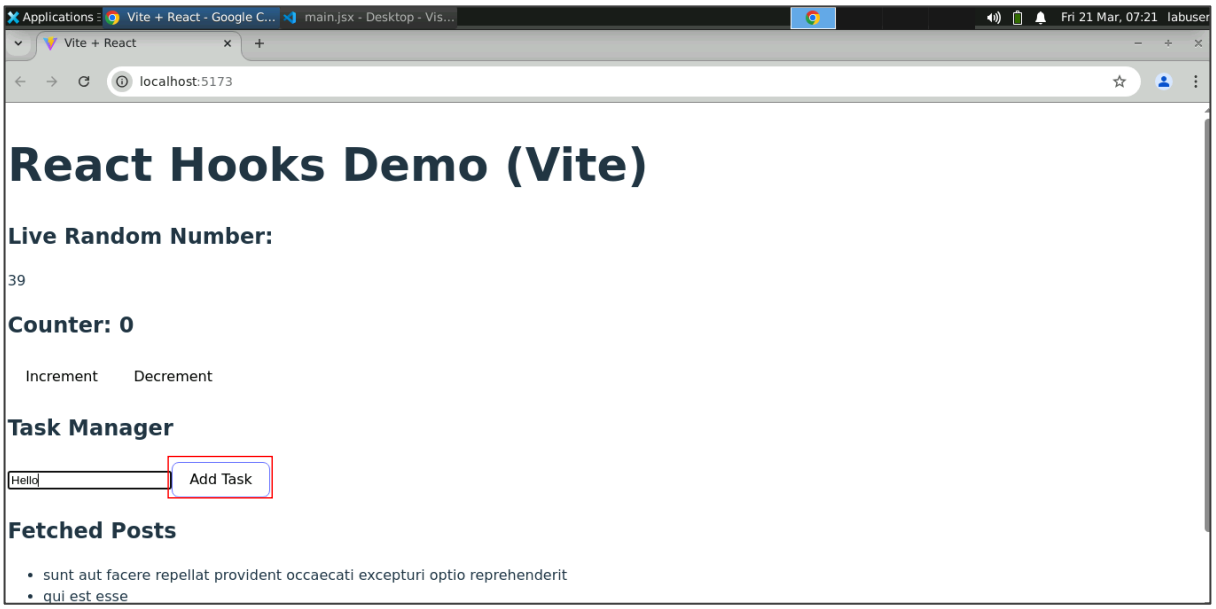


7.3 Click **Decrement** to decrease the count by 1

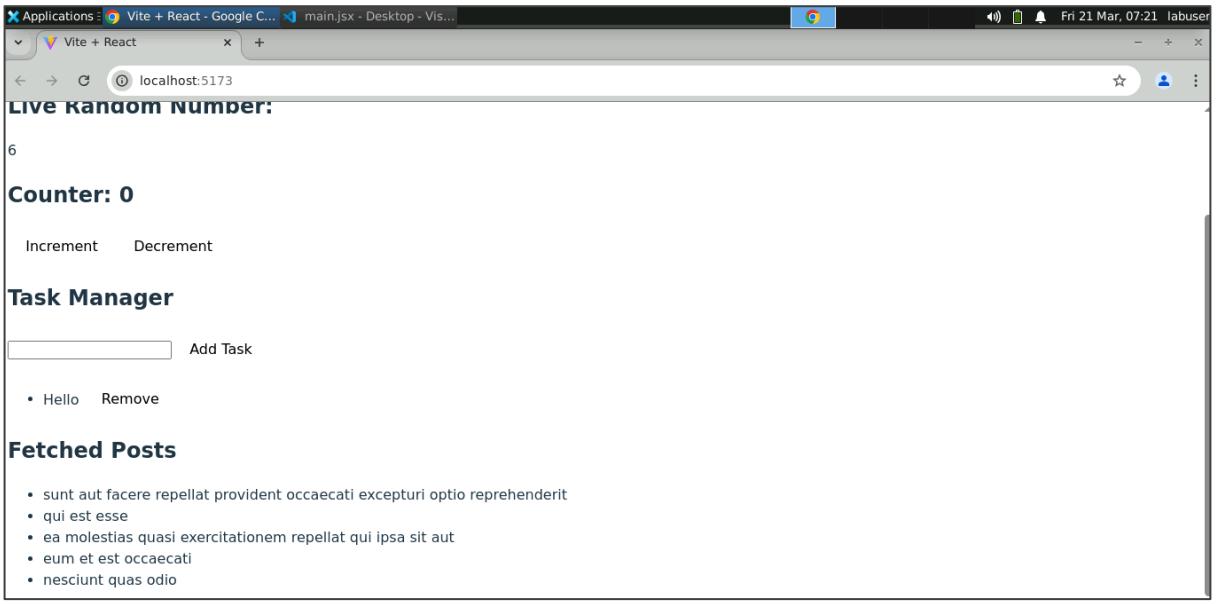




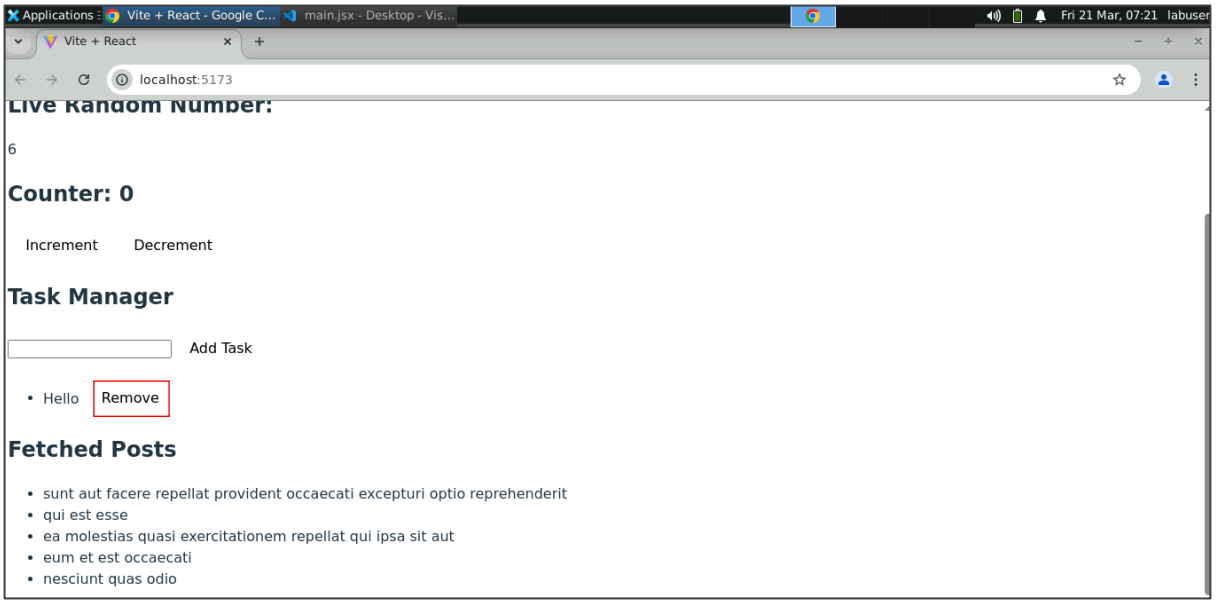
7.4 Enter your preferred task and click **Add Task**



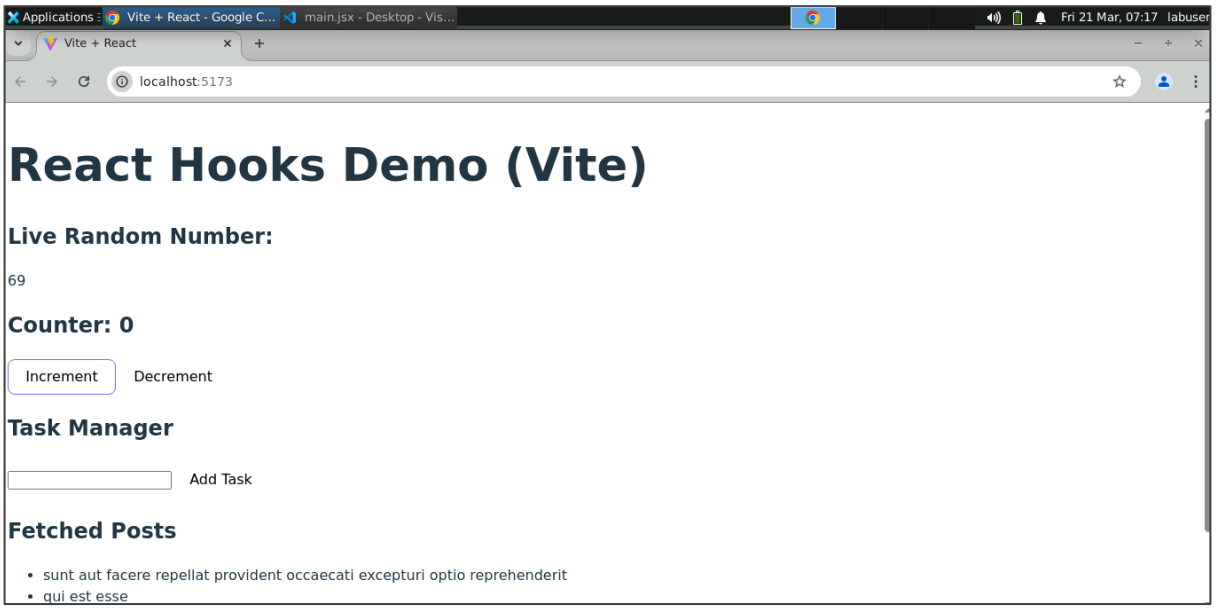
You will see a new Task added to the list.



7.5 Click **Remove** to delete the task that was created



You will see a task deleted from the list.



Note: The Fetched Posts section fetches and displays post titles from https://jsonplaceholder.typicode.com/posts?_limit=5 using a custom useFetch hook with useEffect and useState.

By following the above steps, you have successfully built a modular React application using Vite that demonstrates all major React hooks in action. The project showcases state management with useState and useReducer, side effects with useEffect, global state sharing using useContext, and reusable logic through custom hooks.